Assignment: Composite Design Pattern

Searchable File System

Scenario

Consider the file system of an operating system. It has two types of objects: **File** and **Folder**. In some cases a File and a Folder must be treated in the same way. For example, a user wants to search for a particular keyword, and the search applies to both:

- For a **File**, the search looks into the contents of the file.
- For a **Folder**, the search goes through all the files in the hierarchy under that folder (including sub-folders) to find the keyword.

The client code should be able to call `search(keyword)` on a File, a Folder or the root of the whole file system in exactly the same way, without checking what type of object it is.

Requirements

- A File has a name, a size in KB and text content.
- A Folder has a name and can contain Files and other Folders, nested to any depth. A Folder may be empty.
- Every item supports these operations:
  - `getName()`
  - `getSize()` (a Folder's size is the sum of everything below it)
  - `display()` (prints the item; a Folder prints its children as an indented tree)
  - `search(keyword)` (returns the files whose content contains the keyword, ignoring case)
- A Folder also supports `add(item)` and `remove(item)`.

Tasks

Task 1 – Identify the pattern elements (20% marks)

Name the design pattern that fits this scenario and say why. Then fill in this table for your design:

Pattern role	Your class/interface	Responsibility
Component	?	?
Leaf	?	?
Composite	?	?
Client	?	?

In a few sentences, explain how `search(keyword)` on a Folder works through its children.

Task 2 – Design the UML diagram (30% marks)

Draw a **UML class diagram** (by hand or with a tool such as draw.io or PlantUML) showing all classes and interfaces, their attributes and operations, the relationships between them, and the multiplicity of the Folder's children (`0..*`). The client should depend only on the Component.

Task 3 – Implement the design (50% marks)

Implement your design in any object-oriented language approved by your instructor. Your program must:

1. Use one common Component type for File and Folder.
2. Build the sample file system below and demonstrate every operation.
3. Run `search("design")` on the root, on the `docs` folder and on a single file, using the same client code each time.
4. Add a new file to a folder and show that the folder sizes and search results update.
5. Avoid `instanceof` or type-casting in your logic.

Sample Data

```
root (Folder)
├── docs (Folder)
│   ├── readme.txt   4 KB  "Welcome to the design notes."
│   ├── notes.txt    3 KB  "Composite treats files and folders uniformly."
│   └── reports (Folder)
│       └── q1.txt  10 KB  "Design review scheduled."
├── Main.java        6 KB  "composite demo"
└── todo.txt         1 KB  "Finish design document."
```

Use these values to check your program:

Query	Expected result
`root.getSize()`	24 KB
`docs.getSize()`	17 KB
`root.search("design")`	readme.txt, q1.txt, todo.txt
`docs.search("design")`	readme.txt, q1.txt
`root.search("composite")`	notes.txt, Main.java
`todo.txt.search("design")`	todo.txt
`todo.txt.search("sales")`	no results

Deliverables and Demonstration

Submit:

1. A short document with your answers to Task 1 and your UML class diagram from Task 2.
2. Your source code (Task 3).

You will **demonstrate your program in class by running the code** and briefly explain your design. A program that cannot be run in class receives no marks for Task 3.